

W09D06 — Dockerfile untuk Project 1

ML Engineer Journey | Phase 3 — Docker & Clean Architecture | 10 May 2026

1. Konteks — Synthesis Day

W09D06 adalah hari terakhir Week 9 — bukan hari belajar konsep baru, melainkan hari di mana semua yang dipelajari dari D01 hingga D05 dieksekusi menjadi satu artefak nyata: Dockerfile production-ready untuk Risk Scoring API (Project 1). Ini adalah pola yang akan berulang di roadmap ini — setiap minggu ditutup dengan delivery konkret yang masuk ke repository.

Analoginya seperti memasak untuk restoran — bukan untuk diri sendiri. Dapur boleh berantakan kalau hanya untuk keperluan sendiri. Tapi untuk restoran, setiap langkah harus efisien, bahan tidak perlu tidak ada, dan hasilnya bisa direproduksi oleh siapapun. Dockerfile production-ready adalah 'dapur restoran' — bukan sekadar jalan, tapi dapat diandalkan dan direproduksi di mesin manapun.

2. Development vs Production-Ready Dockerfile

Perbedaan utama bukan di 'apakah jalan', tapi di efisiensi, keamanan, dan reproduktivitas:

Aspek	Development / Asal Jalan	Production-Ready
Base image	python:3.11 (~1.2GB+)	python:3.11-slim (~150MB)
pip install	Tanpa flag	--no-cache-dir (buang cache)
Urutan COPY	COPY . . dulu	requirements.txt dulu (layer cache)
.dockerignore	Tidak ada	Wajib ada
CMD format	Shell form: CMD uvicorn...	Exec form: CMD ["uvicorn", ...]
Ukuran final	~1.2 GB+	~390 MB

3. Layer Caching — Mekanisme Kritis

Docker menyimpan setiap instruksi sebagai layer. Saat build, Docker membandingkan checksum instruksi dan file yang di-COPY dengan cache yang tersimpan. Kalau cocok, layer itu di-skip dan dipakai langsung dari cache.

Yang paling penting: invalidasi bersifat CASCADE. Kalau satu layer berubah, semua layer di bawahnya ikut dieksekusi ulang meski tidak berubah.

Layer	Instruksi	Frekuensi berubah	Behaviour
1	FROM python:3.11-slim	Hampir tidak pernah	Pull dari Hub, cached lama
2	WORKDIR /app	Tidak pernah	Nyaris selalu cached
3	RUN apt-get + clean	Sangat jarang	Cached lama
4	COPY requirements.txt .	Saat tambah dependency	Trigger reinstall pip
5	RUN pip install	Ikut layer 4	Layer paling berat (~240MB)
6	COPY . .	Setiap perubahan kode	Tidak cached, tapi cepat

Itulah kenapa build pertama makan 193 detik, build kedua hanya 1.5 detik — seluruh 6 layer di-cache karena tidak ada perubahan apapun.

4. Dockerfile Production-Ready — Anotasi

Berikut Dockerfile lengkap dengan penjelasan tiap instruksi:

```
# Base image – slim bukan full, selisih ~870MB
FROM python:3.11-slim

# Semua perintah berikutnya relatif ke /app
WORKDIR /app

# System deps dalam 1 RUN – kalau dipisah, apt cache committed di layer sebelumnya
RUN apt-get update \
    && apt-get install -y --no-install-recommends libgomp1 \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*

# requirements.txt DULU – layer cache pattern
COPY requirements.txt .

# --no-cache-dir: buang cache pip, dead weight di image
RUN pip install --no-cache-dir --upgrade pip \
    && pip install --no-cache-dir -r requirements.txt

# Source code di-copy SETELAH pip install
COPY . .

# Deklarasi port – informatif, tidak otomatis publish
EXPOSE 8000

# Exec form – uvicorn jadi PID 1, SIGTERM diterima langsung
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

5. Exec Form vs Shell Form — Signal Handling

Ini sering diabaikan tapi berdampak besar di production. PID 1 adalah process yang menerima sinyal dari kernel — termasuk SIGTERM (perintah graceful shutdown).

	Shell Form	Exec Form
Syntax	CMD uvicorn app.main:app	CMD ["uvicorn", "app.main:app"]

PID 1	/bin/sh	uvicorn (langsung)
SIGTERM	Tidak diteruskan ke app	Diterima langsung oleh uvicorn
Graceful shutdown	Tidak — container di-kill paksa	Ya — request selesai dulu
Production standard	Tidak direkomendasikan	Direkomendasikan

Di log container kamu terlihat 'Started server process [1]' — ini konfirmasi bahwa exec form bekerja benar dan uvicorn berjalan sebagai PID 1.

6. .dockerignore — Kenapa Wajib Ada

Tanpa .dockerignore, perintah COPY . . menyalin seluruh working directory ke image termasuk venv/ (ratusan MB), .git/ (seluruh history commit), dan .env (credentials). Ini bukan hanya pemborosan — ini security risk. File .env yang masuk image bisa dibaca oleh siapapun yang punya akses ke image tersebut.

```

venv/ .venv/ env/          # Virtual environment
__pycache__/ *.py[cod]     # Python bytecode cache
.env *.env                 # JANGAN PERNAH masuk image
.git/ .gitignore           # Git history
.vscode/ .DS_Store         # Editor dan OS artifacts
notebooks/ *.ipynb         # Eksperimen, bukan production code
*.csv *.xlsx               # Dataset
.pytest_cache/             # Test cache

```

7. Kapan .env Dibutuhkan

.env adalah file konfigurasi yang menyimpan informasi sensitif atau environment-specific terpisah dari kode. Analoginya: kunci brankas yang disimpan terpisah dari brankas itu sendiri.

Kondisi	Contoh	Wajib .env?
Ada database	DATABASE_URL=postgresql://...	Ya
Ada API key eksternal	OPENAI_API_KEY=sk-...	Ya
Ada JWT secret	SECRET_KEY=abc123...	Ya
Multi-environment	Local vs Railway vs staging	Ya
Semua hardcoded, tanpa credentials	Seperti Risk Scoring API sekarang	Tidak wajib

■ Yang di-commit ke Git bukan .env, melainkan .env.example — berisi key tanpa value. .env selalu ada di .gitignore dan .dockerignore.

8. Hasil Verifikasi — Risk Scoring API

Semua tahap berhasil dieksekusi dan diverifikasi:

Tahap	Hasil
docker build (pertama)	193.7s — semua layer dieksekusi
docker build (kedua)	1.5s — semua 6 layer CACHED

docker run	Container ID: 629d3e2f2952
docker ps	STATUS: Up, PORT: 0.0.0.0:8000->8000/tcp
docker logs	Uvicorn running on http://0.0.0.0:8000, PID 1
Model load	pipeline_pipe_v1_20260415.joblib loaded successfully
GET /health	{"status": "ok", "version": "1.0.0"}

9. Quick Reference — Docker Commands

```

docker build -t risk-scoring-api:v1 .          # build image
docker run -d -p 8000:8000 --name risk-api risk-scoring-api:v1
docker run -d -p 8000:8000 --env-file .env --name risk-api risk-scoring-api:v1
docker ps                                     # container running
docker logs risk-api                         # lihat log
docker rm -f risk-api                       # stop & hapus container
docker images risk-scoring-api              # cek ukuran image
docker build -t risk-scoring-api:v1 . --no-cache # force rebuild

```

10. Koneksi ke Materi Selanjutnya

W09D06 menutup Week 9 — seluruh Docker fundamentals sudah dikuasai: layer caching, image optimization, slim base image, .dockerignore, exec form CMD, dan sekarang Dockerfile production-ready untuk project nyata.

Multi-stage build — cara satu-satunya untuk mengurangi ukuran image lebih jauh dari 390MB (batas yang bisa dicapai dengan single-stage) — dijadwalkan di W12D01. Untuk sekarang, 390MB adalah ukuran optimal yang bisa dicapai selama numpy, scikit-learn, dan FastAPI masih menjadi dependencies.